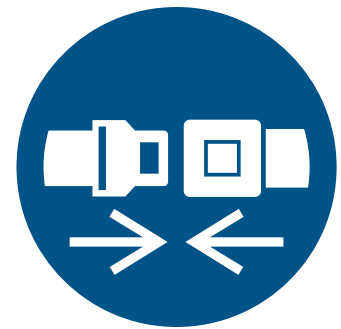
AutoOMPPass

Automatic Parallelization of Loops in LLVM



READY

[00] Contents

01 Motivation

02 Design

03 Live demo

04 Correctness & Performance

[01] Motivation

Writing parallel code is hard:

- You must guarantee **no data races**
- You must reason about **aliasing**
- You must handle **reductions correctly**

Mistakes don’t crash, they silently corrupt results.

Meanwhile, the compiler already knows:

- Memory dependencies
- Loop bounds
- Alias relationships



[01] Why Not Polly?

> *"Polly uses a mathematical representation, the polyhedral model... to apply advanced automatic parallelization."*
>
> — POLLY.LLVM.ORG

Instead of relying on the Control Flow Graph (CFG), Polly converts code into **linear inequalities**.

Polly Rejects:

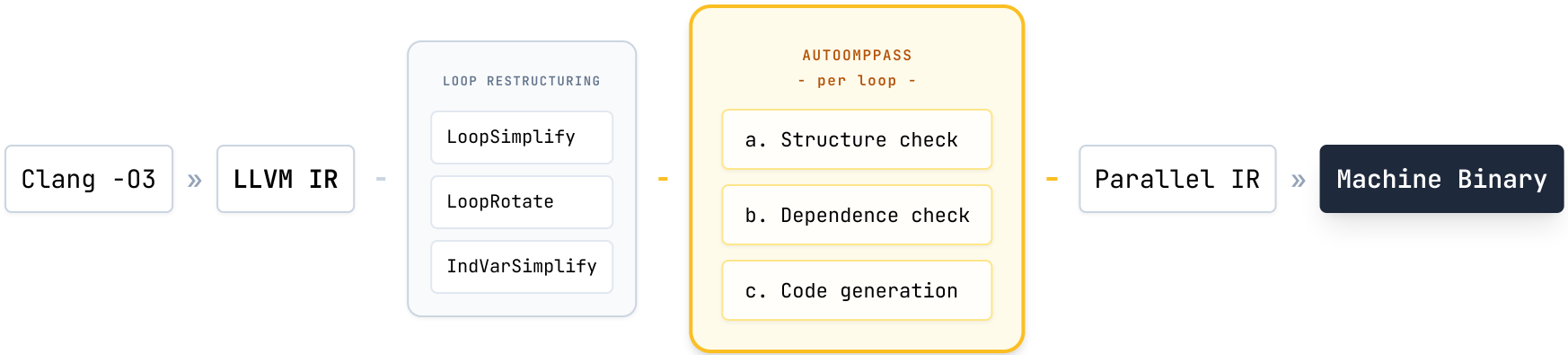
- **Opaque Function Calls**
- **Conditionals**
- **Non-affine indexing**

AutoOMPPass Accepts:

- **Pure Calls** via `readnone` attributes
- **Reductions**

By analyzing the `CFG` and LLVM Attributes directly, we are significantly **less restrictive**.

[02] Design


Clang -O3
LLVM IR
LOOP RESTRUCTURING
LoopSimplify
LoopRotate
IndVarSimplify
AUTOOMPPASS
- per loop -
a. Structure check
b. Dependence check
c. Code generation
Parallel IR
Machine Binary

[02.a] Structural Analysis

The pass must verify that the loop follows a canonical form that the OpenMP accepts.

STRUCTURAL CHECKS

- **Invariance:** The loop bounds and step size must be loop-invariant.
- **Single Entry/Exit:** Ensure a strictly structured CFG with no early exits.
- **Heuristic Filtering:** The Trip Count must be determinable and > 4096.
- **IO Bottleneck Detection:** Reject loops where memory ops dominate arithmetic — threading overhead would outweigh any benefit.

```
# clang++ -Rpass-analysis=auto-omp
remark: rejected; loop contains multiple exit blocks [early_exit]
remark: rejected; trip count is below parallelization threshold (4096)
remark: rejected; loop is memory-bound (IO bottleneck)
```

[02.b] Dependence Analysis

Before transformation, the pass must prove that no data flow exists between iterations.

DEPENDENCE CHECKS

- **Check reductions:** Verify that PHI-node cycles are strictly associative and commutative.
- **Liveness analysis:** Identify loop-local `alloca` instructions to ensure they can be safely **privatized**.
- **Memory independence:** Use `DependenceInfo` to prove pointer-based stores (arrays) don't overlap.
- **Side-effect:** Reject loops containing opaque function calls or `volatile` memory accesses. Apply inter-procedural checks, inspect the function body for side effects.
- **Concurrency:** Reject loops writing to atomics.

```
# clang++ -Rpass-analysis=auto-omp
remark: ❌ rejected; potential loop-carried dependency detected
remark: ❌ rejected; call to 'printf' may have global side effects
```

[02.c] Code Generation

Unlike `#pragma omp` which handles the heavy lifting in source code, `OMPIRBuilder` requires manual orchestration of the LLVM IR transformation.

TRANSFORMATION STEPS

- **Privatization:** Isolate loop-locals and allocate reduction accumulators.
- **Body Cloning:** Create another loop with remapped variables.
- **CFG Cleanup:** Delete the original serial loop.
- **Result Mapping:** Connect external users to the final merged reduction values.

```
# clang++ -Rpass-analysis=auto-omp
remark: ✅ parallelized: #pragma omp parallel for reduction(+:x)
remark: ✅ parallelized: #omp parallel for
```

[02.c] Loop Collapsing

Collapse perfectly nested loops.

COLLAPSE REQUIREMENTS

- **Perfectly nested:** Only terminators and PHI nodes allowed in outer loop blocks.
- **Rectangular iteration space:** Inner bounds must be **invariant** w.r.t. outer induction variables.
- **IV derivation:** Individual IVs are recovered from the collapsed index via `div` / `rem`.

```
# clang++ -Rpass-analysis=auto-omp
remark: ✅ parallelized: #pragma omp parallel for collapse(2)
remark: rejected; non-rectangular iteration space — inner bound depends on outer IV
```

[02] Walkthrough: Input

ORIGINAL C CODE

```c
double dot(double *a, double *b, int n) {
    double sum = 0.0;
    int tmp;
    for (int i = 0; i < n; i++) {
        tmp = i * 2;
        sum += a[i] * b[tmp];
    }
    return sum;
}
```

```
$ clang -O3 -S -emit-llvm dot.c # produces LLVM IR
$ opt -load-pass-plugin=AutoOMPPass.so -passes=auto-omp dot.ll
```

[02] Walkthrough: Loop Restructuring

AFTER LOOPROTATE + INDVARSIMPLIFY

The prerequisite passes normalize the loop into a canonical form the pass can reason about.

BEFORE (while-style, top-tested)

```
// top-tested: condition checked first
i = 0;
while (i < n) {
    tmp = i * 2;
    sum += a[i] * b[tmp];
    i++;
}
```

AFTER (do-while, canonical IV)

```
// guard + rotated loop
if (n > 0) {
  i = 0;        // canonical IV start
  do {
    tmp = i * 2;
    sum += a[i] * b[tmp];
    i++;
  } while (i < n); // back-edge
}
```

[02] Walkthrough: Structure Check

VERIFY PARALLELIZATION PREREQUISITES

```
double dot(double *a, double *b, int n) {
    double sum = 0.0;
    int tmp;
    for (int i = 0; i < n; i++) {
        tmp = i * 2;
        sum += a[i] * b[tmp];
    }
    return sum;
}
```

- ✓ Trip bounds: all loop-invariant
- ✓ Single entry / single exit — no `break` , `goto` , or early `return`
- ✓ Trip count `n` — assumed `> 4096` at runtime (heuristic threshold)
- ✓ Not IO-bound — `2 arith ops > 0 mem ops` inside body

[02] Walkthrough: Dependence Check

IDENTIFY VARIABLES AND PROVE INDEPENDENCE

```
double dot(double *a, double *b, int n) {
    double sum = 0.0;
    int tmp;
    for (int i = 0; i < n; i++) {
        tmp = i * 2;
        sum += a[i] * b[tmp];
    }
    return sum;
}
```

Loop-local (privatize)

`tmp` — written each iteration, never read across iterations

Reduction (PHI cycle)

`sum` — `φ(0.0, sum + ...)` associative & commutative (`fadd`)

Read-only (safe)

`a[i]` , `b[tmp]` — no stores to `a` or `b` , no aliasing

```
remark: ✅ all checks passed — loop is safe to parallelize
```

[02] Walkthrough: Thread Execution

PARALLEL EXECUTION — EACH THREAD GETS PRIVATE COPIES

The loop range is split across threads. Each thread receives its own private `tmp` and local `sum`.

Thread 0

```
double sum_local = 0.0;
int tmp_private;
for (i = 0; i < n/3; i++) {
  tmp_private = i * 2;
  sum_local += a[i] * b[tmp_privat
}
```

Thread 1

```
double sum_local = 0.0;
int tmp_private;
for (i = n/3; i < 2*n/3; i++) {
  tmp_private = i * 2;
  sum_local += a[i] * b[tmp_privat
}
```

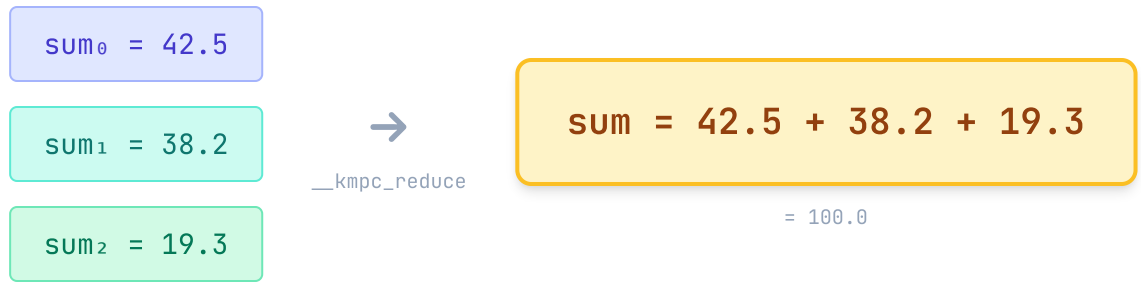
Thread 2

```
double sum_local = 0.0;
int tmp_private;
for (i = 2*n/3; i < n; i++) {
  tmp_private = i * 2;
  sum_local += a[i] * b[tmp_privat
}
```

Each thread writes only to its **private** variables.

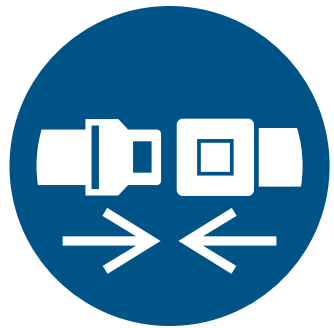
[02] Walkthrough: Reduction Merge

AFTER ALL THREADS COMPLETE — COMBINE PARTIAL RESULTS




```
// Generated equivalent — what AutoOMPPass produces
#pragma omp parallel for reduction(+:sum) private(tmp)
for (int i = 0; i < n; i++) {
    tmp = i * 2;
    sum += a[i] * b[tmp];
}
// sum now holds the final merged value
```

[03] Live Demo



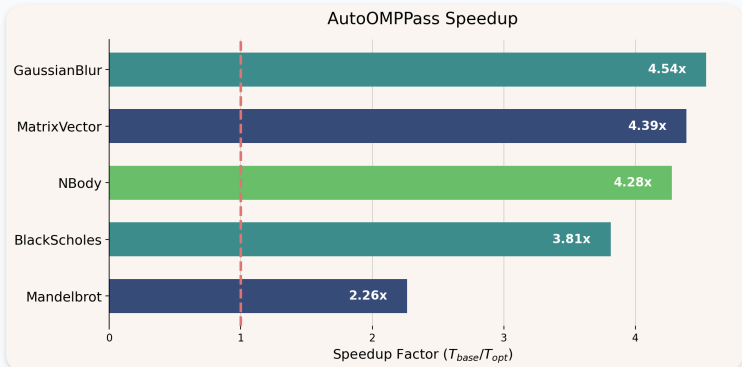
CHECKING...

[04] Correctness & Performance

TSVC 2 Test Suite

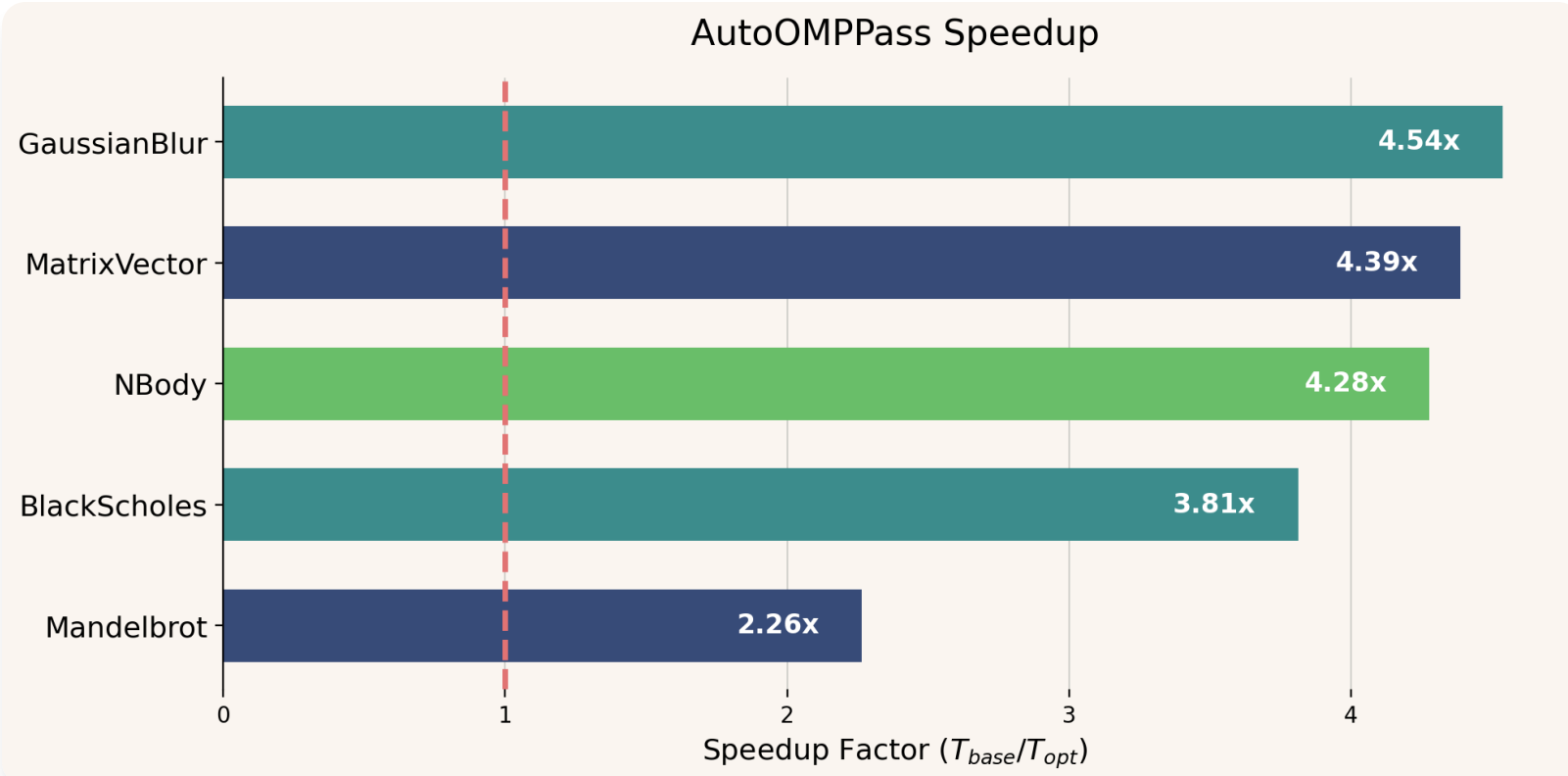
Correctness validated against the **Test Suite for Vectorizing Compilers** (TSVC). Standard suite covering reductions, loop-carried deps, and aliasing patterns over **147** tests.

Benchmarks



N = 20M elements · median of 5 runs · -O3

[04] Benchmarks



N = 20M elements · median of 5 runs · -O3

EOF (End of File)

Questions?

process exited with status 0